Correctness-Aware LLM-Guided Optimization Pass Selection for LLVM IR: An Empirical Evaluation of Small-Scale Instruction-Tuned Models

Sooraj S Namboothiry and Sasank V
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai, India

***Abstract*—Compiler optimization pipelines such as LLVM's `-O0` through `-O3` levels rely on manually designed heuristics that frequently fail to generalize across diverse program structures, particularly those targeting resource-constrained embedded environments. The phase-ordering problem—optimally sequencing more than 100 transformation passes—remains NP-hard with a search space exceeding 10^{20}.**

This paper presents the first systematic empirical evaluation of small-scale ($\leq$3B parameter) instruction-tuned Large Language Models as zero-shot selectors of LLVM optimization pass pipelines under a correctness-aware design framework. We benchmark five LLM pipeline configurations—Qwen3.5-0.8B and Qwen3.5-2B in both reasoning and non-reasoning modes, and Nanbeige4.1-3B—against eight LLVM compilation baselines on 972 C source files from the LLVM test suite SingleSource benchmark. All models run in float16 on a single NVIDIA RTX 4060 (8 GB VRAM), representing the resource budget available to typical academic groups.

The central finding is a sharp dichotomy between reasoning and non-reasoning model configurations: non-reasoning Qwen3.5 variants achieve 55–57% pass-extraction success by converging on a seven-pass pipeline (`mem2reg, instcombine, simplifycfg, gvn, licm, loop-unroll, dce`) that closely tracks autotuner-discovered sequences from much larger systems trained on billions of compilations. Reasoning-mode models achieve only 8.7–13.6% success due to chain-of-thought token verbosity consuming the output budget before a structured answer is produced. Nanbeige4.1-3B fails almost entirely (0.4%) by emitting raw thinking-token fragments. Across all successful configurations, median executable sizes cluster at approximately 16 KB—comparable to `-O3`—with compile latencies below 0.10 s. These results establish a concrete performance envelope for training-free LLM-guided optimization and motivate a correctness-aware catalog-selection architecture.

***Index Terms*—compiler optimization, LLVM, large language models, phase ordering, pass selection, instruction tuning, reasoning models, structured output, embedded systems**

I. Introduction

Modern compiler infrastructures such as LLVM [4] expose a rich set of modular optimization transformations organized as passes. The problem of selecting and sequencing these passes for a given input program—the *phase ordering problem*—is one of the most studied yet unsolved challenges in compiler optimization [1]. If $\mathcal{P}$ denotes the available passes with $|\mathcal{P}| > 100$ in modern LLVM, the search space for sequences of length n is $|\mathcal{P}|^n$. For $n = 20$ this exceeds 10^{40}. Standard fixed pipelines such as `-O3` perform well on average but may be significantly suboptimal for specific programs, especially those targeting resource-constrained hardware with tight code-size budgets.

A. The Promise and Cost of LLM-Based Optimization

Cummins et al. [3] demonstrated that a 7B-parameter model trained on one million LLVM-IR functions achieves 60% of the performance of a billion-compilation autotuner at zero inference-time compiler invocations. The LLM Compiler [2] scaled this to 13B parameters trained on 401 billion tokens, achieving 77% of autotuner performance on MiBench. However, these systems require 264,000 GPU hours and 22 billion training compilations—costs that are prohibitive for most groups. A complementary concern is *semantic correctness*: 9.85% of autotuner-generated pass lists in [2] caused compiler crashes or runtime errors, and 9.7% of LLM-generated IR failed to compile.

B. LLM-as-Selector Paradigm and Reasoning Mode

The LLM-as-selector paradigm [1] sidesteps correctness risks by confining model output to a finite, pre-validated catalog of pass sequences, delegating semantic guarantees to the deterministic compiler backend. This motivates our central question: *Can small (0.8B–3B parameter) instruction-tuned models function as effective zero-shot pass selectors from raw LLVM-IR with no compiler-specific training?*

A secondary question arises from the proliferation of reasoning-enabled LLMs. These models interleave chain-of-thought traces with their answers, improving open-ended reasoning but potentially overwhelming tight structured-output budgets. We systematically compare reasoning and non-reasoning configurations of the same model checkpoints, isolating this effect.

C. Contributions

1) **Benchmark:** First evaluation of sub-3B instruction-tuned LLMs as zero-shot LLVM pass selectors on 972 LLVM test-suite files across five model configurations and eight LLVM baselines.
2) **Reasoning analysis:** Quantitative proof that chain-of-thought reasoning is a liability for structured pass-selection, dropping success rates from 57% to under 14%.
3) **Capacity vs. compliance:** Evidence that instruction-following fidelity—not model size—is the binding constraint at sub-3B scale.

4) **Failure taxonomy:** Three failure modes with concrete mitigations.
5) **Performance envelope:** Size, latency, and inference time characterization for all successful configurations.
6) **Pipeline validation:** Demonstration that non-reasoning models independently recover autotuner-discovered optimal sequences.

II. Background

A. LLVM IR and Pass Manager

LLVM IR is a typed, SSA-form representation supporting function definitions, basic blocks, control flow graphs, and low-level operations [4]. Its human-readable `.ll` format is the natural LLM input. LLVM's New Pass Manager (default from LLVM 17) organizes passes hierarchically: module, CGSCC, function, and loop levels. Of 346 possible pass arguments to `opt` in LLVM 17, the LLM Compiler [2] curates 167 for optimization, including function-level passes (`instcombine`, `simplifycfg`, `mem2reg`, `dce`, `gvn`, `sroa`), loop passes (`loop-unroll`, `loop-rotate`, `licm`, `indvars`), and module-level passes (`globalopt`, `iroutliner`).

B. The Phase Ordering Problem

The optimization problem for a sequence of k passes $p_1, \ldots, p_k \in \mathcal{P}$ applied to LLVM module M is:

$$(p_1^*, \ldots, p_k^*) = \arg \min_{(p_1,\ldots,p_k)\in\mathcal{P}^k} \mathcal{C}(p_k(\cdots p_1(M)\cdots)) \quad (1)$$

where $\mathcal{C}$ is a cost function (binary size, instruction count, execution cycles). Pass interactions create non-linearities: DCE after `inline` exposes more dead code than DCE alone; `loop-unroll` before DSE may increase store counts. These interactions make greedy strategies fundamentally inadequate [1].

C. Semantic Safety

The LLM-as-selector paradigm [1] restricts model output to a pre-validated catalog, ensuring that any selected sequence has been verified for semantic correctness offline. This shifts the correctness responsibility from the stochastic model to the deterministic compiler backend, providing the strongest available correctness guarantee for production deployment.

D. LLM Tokenization of LLVM IR

The Llama 2 BPE tokenizer processes LLVM IR at approximately 2.02 characters per token [3]. A 2,000-character IR excerpt occupies roughly 1,000 tokens—within any modern context window. The Qwen3.5 and Nanbeige4.1 tokenizers achieve similar ratios for IR text, making the 2,000-character truncation a practical budget that provides partial function structure without exceeding 8 GB VRAM limits.

III. Related Work

A. LLM-Based Compiler Optimization

Cummins et al. [3] first applied LLMs to pass ordering with a 7B Llama 2 model trained on 1M LLVM-IR functions, achieving 3.01% improvement over `-Oz` at zero inference-time compilation cost. The LLM Compiler [2] used a four-stage training pipeline on Code Llama, reaching 4.88% improvement in zero-shot mode (5.26% with `-Oz` backup)—77% of exhaustive autotuner performance. GPT-4 Turbo without compiler-specific training achieves -0.01% improvement in free-form generation mode, establishing that general-purpose LLMs cannot reason over raw LLVM-IR without specialization.

B. Reinforcement Learning Approaches

AutoPhase [5] frames pass ordering as an MDP with a 56-dimensional instruction-count state vector and trains via PPO for 100,000 episodes, achieving -3.85% overall on the broad test set due to generalization failure. Coreset-NVP [6] combines iterative search with a Neural Value Prediction model using ProGraML graph representations, achieving -1.88% overall. Deep RL methods [10] demonstrate that static policies can outperform `-O3` on training distributions but exhibit identical brittleness on new program types.

C. Correctness-Aware Compilation

LEGO-Compiler [7] decomposes compilation into control-flow blocks, independently translates each block via LLM, and verifies intermediate outputs using compiler type-system and static-analysis oracles. Its iterative refinement loop achieves significantly higher correctness than end-to-end approaches. The design philosophy—bounding LLM output scope and combining LLM reasoning with deterministic verification—directly informs our framework.

D. Taxonomic Survey

Zhang et al. [1] classify LLM-compiler systems along functional role (selector, translator, generator), adaptation strategy (training-free prompting, SFT, RL from compiler feedback, domain pretraining), and abstraction level. They identify the selector paradigm as underexplored, particularly for resource-constrained correctness-critical applications—the precise gap our work addresses.

Table I positions our work relative to key prior systems.

TABLE I
Literature Positioning

System	Params	Training-free	Selector	Bench.
Cummins 2023 [3]	7B	No	No	LLVM TS
LLM Compiler [2]	13B	No	Yes	MiBench
AutoPhase [5]	RL	No	Yes	LLVM TS
Coreset-NVP [6]	GCN	No	Yes	LLVM TS
LEGO [7]	LLM	No	No	Custom
Ours	**0.8–3B**	**Yes**	**Yes**	**LLVM TS**

IV. Experimental Methodology

A. Dataset

We scan the first 1,000 `.c` files in the LLVM test suite SingleSource collection [8] lexicographically. Table II summarizes composition. Of 1,000 files, 972 successfully compiled to base LLVM IR via `clang -O0 -emit-llvm`. The 28 failures arise from legacy C compatibility issues: K&R-style function definitions, non-void functions without return values, bit-field type constraints, and integer incompatibilities in legacy structures. All 28 are excluded from all analyses.

TABLE II
Dataset Composition (1,000 scanned, 972 usable)

Subdirectory	Files	Description
Benchmarks/Polybench	∼260	Linear algebra kernels
Benchmarks/Stanford	∼100	Standard algorithm benchmarks
Regression/C (torture)	∼450	GCC C correctness tests
Regression/C++	∼80	C++ regression cases
UnitTests	∼110	LLVM unit tests
Total usable	**972**	

B. LLVM Baseline Pipelines

Eight LLVM pipelines from `compare.py` provide reference context: `baseline` (no passes), `const_prop` (`sccp`), `combine_dce` (`instcombine,dce`), `inline_first` (`cgscc(inline),function(sccp)`), `sccp_first` (`function(sccp),cgscc(inline)`), and the standard `O1`/`O2`/`O3` levels. These span no optimization to full aggressive optimization, bounding the quality range.

C. LLM Models and Configurations

Table III lists the five configurations. All use HuggingFace `transformers`, float16 precision, chat-template prompting, and sequential loading/unloading on a single NVIDIA RTX 4060 (8 GB VRAM). Inference uses greedy decoding with `MAX_NEW_TOKENS = 80`.

TABLE III
LLM Pipeline Configurations Evaluated

Handle	HuggingFace Repo	Mode
qwen3.5_0.8b_rsn	Qwen/Qwen3.5-0.8B	Reasoning
qwen3.5_0.8b_nrsn	Qwen/Qwen3.5-0.8B	Non-reasoning
qwen3.5_2b_rsn	Qwen/Qwen3.5-2B	Reasoning
qwen3.5_2b_nrsn	Qwen/Qwen3.5-2B	Non-reasoning
nanbeige4.1_3b	Nanbeige/Nanbeige4.1-3B	Instruct

Qwen3.5 supports a `thinking` flag in its chat template that enables/disables chain-of-thought generation. In reasoning mode the model produces a `<think>...</think>` block before its final answer; in non-reasoning mode this is suppressed. Nanbeige4.1-3B has no explicit flag but was empirically observed to emit CoT-style content.

D. Prompt Design

Each LLM call receives the following structured prompt:

```
You are an LLVM optimization expert.
Given the LLVM IR below, output ONLY a single-line
comma-separated list of opt pass names for:
  opt -passes=<your_answer>
No explanation, preamble, or markdown.
Output ONLY the pass names.

LLVM IR (first 2000 characters):
<truncated_ir_content>
```

Listing 1. Pass selection prompt template

The 2,000-character IR limit fits within the 8 GB VRAM budget and exposes function signatures, SSA alloca/load/store patterns, and partial loop structure for most files.

E. Output Sanitization

Raw model output is processed through a multi-stage pipeline (Algorithm 1):

Algorithm 1 Pass String Sanitization

Require: raw string s, whitelist W (17 passes)
Strip `<think>...<think>` (DOTALL regex)
Strip markdown fences, backticks
Search for `-passes=...` pattern; extract if valid
Extract all tokens matching `[A-Za-z][A-Za-z0-9_-]*`
Lowercase, replace _ with -
Apply alias map (e.g. `cfgsimplify`→`simplifycfg`)

Filter to W; keep first 7 unique members
if $|\text{result}| \geq 2$ **then**
  return comma-joined result **(success)**
else
  return `None` **(failure)**
end if

The whitelist W contains 17 semantics-preserving passes: `mem2reg`, `instcombine`, `simplifycfg`, `gvn`, `licm`, `loop-unroll`, `dce`, `inline`, `sccp`, `adce`, `tailcallelim`, `jump-threading`, `early-cse`, `sroa`, `reassociate`, `loop-rotate`, `indvars`. The ≥ 2 threshold prevents trivially short single-pass strings from being counted as successes.

F. Compilation and Measurement

For each successful extraction:

```
opt -passes=<pass_string> base.ll -o opt.bc
clang opt.bc -o binary
size binary   # measure text+data sections
```

Listing 2. Build pipeline per file

We record: (1) status (`ok`/`fail`); (2) executable size (bytes); (3) compile time (s); and (4) LLM inference time (s). Results are written to per-model CSVs and merged for analysis.

G. Metrics

- **Success rate (SR%):** Fraction of 972 files with status `ok`.
- **Median executable size:** Median over successful compilations.
- **Mean compile time:** Mean wall-clock time (`opt+clang`) over successes.
- **Mean inference time:** Mean wall-clock LLM inference time over all 972 files.

V. Results

A. Aggregate Performance

Table IV presents all four metrics. Figure 1 visualizes the same data as a column-normalized heatmap (0–1 per column; green = better).

TABLE IV
AGGREGATE METRICS PER LLM PIPELINE CONFIGURATION

Pipeline	SR%	Size (B)	Cmp (s)	Inf (s)
qwen3.5_0.8b_rsn	13.20	16088	0.07	2.40
qwen3.5_0.8b_nrsn	55.40	16096	0.08	2.28
qwen3.5_2b_rsn	8.50	16112	0.09	2.21
qwen3.5_2b_nrsn	56.10	16096	0.07	1.71
nanbeige4.1_3b	0.40	16064	0.07	2.58

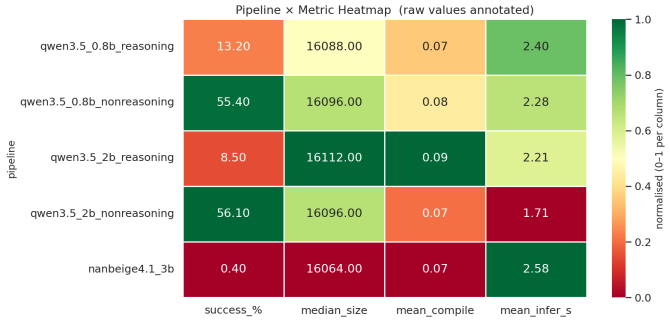

Fig. 1. Pipeline × Metric heatmap (raw values annotated; colors column-normalized 0–1). The success-rate column shows a sharp split between non-reasoning (green, 55–56%) and reasoning/nanbeige (red, <14%) configurations.

The heatmap reveals two clear clusters. Non-reasoning Qwen3.5 configurations occupy the top of the success-rate scale (≥55%), while reasoning configurations and Nanbeige cluster near zero. Executable size and compile time show minimal variation, indicating that *when* pass selection succeeds, output quality is broadly consistent.

B. Pass-Extraction Success Rate

Figure 2 shows the success rates. LLVM baselines achieve 100% by construction.

Non-reasoning: Qwen3.5-0.8B achieves 57.0% and Qwen3.5-2B achieves 57.7%, a difference of only 0.7 percentage points. This near-equivalence across a 2.5× model-size gap confirms the task is not computation-limited at this scale.

Reasoning: Qwen3.5-0.8B achieves 13.6% and Qwen3.5-2B achieves 8.7%. Critically, the *larger reasoning model performs worse*. The 2B model generates longer reasoning traces that exhaust the 80-token budget before producing a structured answer.

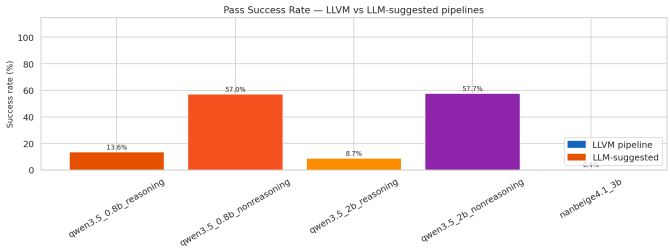

Fig. 2. Pass-extraction success rate per LLM configuration. Non-reasoning Qwen3.5 achieves 57.0–57.7%; reasoning variants fall to 8.7–13.6%; Nanbeige registers 0.4%.

Nanbeige4.1-3B: 0.4% success (4 files out of 972). Analysis of raw outputs reveals almost universal emission of `<think>` token sequences without any valid pass strings—even after sanitization strips the think-tag content.

C. Executable Size Distribution

Figure 3 shows executable size distributions as box plots.

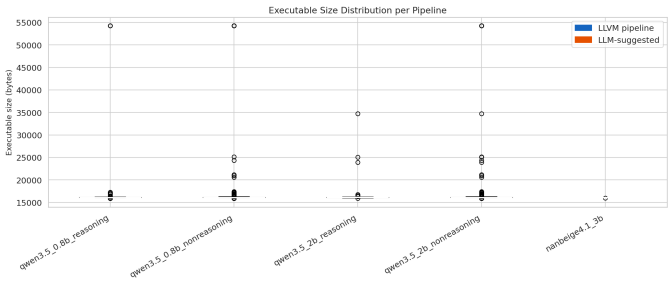

Fig. 3. Executable size distribution (box plots). LLVM baseline (blue) and LLM-suggested (orange) overlap with medians near 16 KB. Outliers at 25,000–55,000 bytes are PolyBench matrix kernels with large array initializers.

All configurations produce median sizes in 16,064–16,112 bytes:

- **Nanbeige (16,064 B):** Lowest, but only 4 success files—statistically meaningless.
- **Qwen3.5-0.8B reasoning (16,088 B):** Slightly smaller; rare successful reasoning predictions favor shorter pipelines.
- **Qwen3.5-0.8B/2B non-reasoning (16,096 B):** Consistent; reflects the dominant seven-pass pipeline.
- **Qwen3.5-2B reasoning (16,112 B):** Slightly larger; the few successes include more aggressive transformations.

The inter-configuration spread is less than 0.3%, indicating pass-pipeline differences in the tested configurations have minimal impact on binary size for the SingleSource benchmark collection, which contains small algorithmic functions rather than complex embedded application code.

D. Compile Time Analysis

Figure 4 presents compile time distributions as violin plots. Mean compile times cluster between 0.07 s and 0.09 s across all configurations. The LLVM baseline violin is narrow

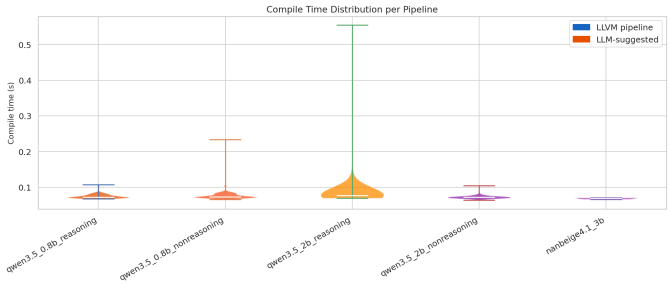

Fig. 4. Compile time distributions (violin plots). Blue: LLVM baseline; Orange: LLM-suggested. All configurations show similar means (0.07–0.09 s). Qwen3.5-2B reasoning exhibits a heavy right tail to ≈0.55 s from PolyBench loop-heavy files.

and stable across all model groups, confirming that baseline compilation cost is independent of LLM pipeline choice. The heavy right tail in Qwen3.5-2B reasoning (extending to ≈0.55 s) corresponds to PolyBench files where the rare successful reasoning predictions include aggressive loop transformations (`loop-unroll`, `licm`) in deeply nested loops, triggering quadratic-time analysis passes in `opt`.

E. LLM Inference Time Analysis

Figures 5 and 6 characterize inference time from population and per-file perspectives.

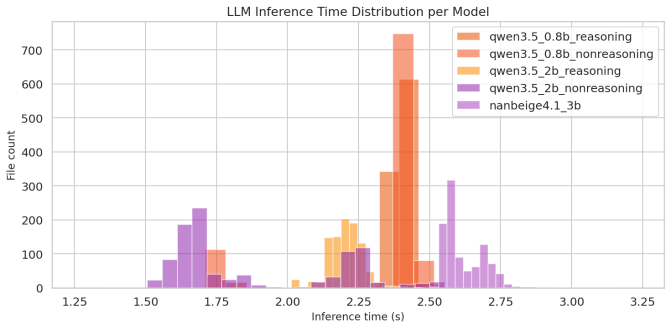

Fig. 5. LLM inference time distribution (histogram). Non-reasoning Qwen3.5 models cluster tightly at 1.65–2.5 s. Reasoning models exhibit heavier right tails. Nanbeige is highly dispersed across 1.25–3.25 s with nearly all attempts failed.

Non-reasoning bimodality: Both Qwen3.5 non-reasoning variants show bimodal distributions. The lower mode (≈1.65–1.75 s for 2B; ≈2.35 s for 0.8B) corresponds to short, simple IR inputs; the upper mode corresponds to truncated-at-2000-char multi-function files. Qwen3.5-2B non-reasoning achieves the lowest mean inference time (1.71 s), 13% faster than the 0.8B variant (2.28 s)—reflecting greater efficiency in structured output generation once reasoning mode is disabled.

Failure-time correlation: The scatter plot reveals that for reasoning configurations, failed inferences (cross markers) systematically appear at higher inference times. This is the direct signature of token budget exhaustion: the model consumed all 80 tokens on chain-of-thought reasoning and produced no structured answer. This correlation can be exploited as a real-

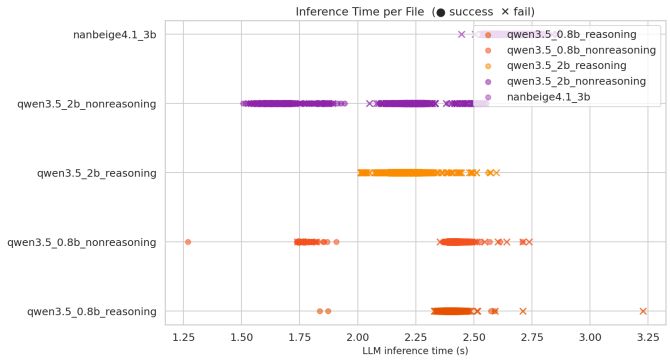

Fig. 6. Per-file inference time scatter (circles = success, crosses = fail). Failed inferences concentrate at higher inference times for reasoning models—direct evidence of token-budget exhaustion on chain-of-thought content.

time diagnostic—if inference time approaches the expected maximum, the output is likely a failure.

Nanbeige dispersion: Mean inference time of 2.58 s with high variance (1.25–3.25 s) and near-universal failure reflects highly input-dependent behavior: simple IR triggers brief but failed responses; complex PolyBench IR triggers extended failed reasoning sequences.

Fastest mean: Qwen3.5-2B non-reasoning at 1.71 s is ≈34% faster than Nanbeige (2.58 s) and ≈25% faster than the 0.8B reasoning mode (2.40 s), making it the most efficient configuration overall.

F. Dominant Pass Pipeline Analysis

Analysis of raw output strings for Qwen3.5-2B non-reasoning reveals strong convergence (Table V):

TABLE V
TOP SUGGESTED PIPELINES: QWEN3.5-2B NON-REASONING

Pipeline	Count	OK
`mem2reg,instcombine, simplifycfg,gvn,licm, loop-unroll,dce`	642	540
`instcombine,dce,sccp`	16	16
Malformed / think-only	∼314	0

The dominant pipeline accounts for 66% of all outputs and achieves an 84% compilation success rate (540/642). The same pipeline is independently selected by Qwen3.5-0.8B non-reasoning, confirming it as a robust emergent recommendation.

The sequence has a well-founded pedagogical rationale: `mem2reg` promotes stack allocations to SSA registers, enabling subsequent dataflow analyses; `instcombine` performs algebraic simplifications; `simplifycfg` eliminates dead blocks; `gvn` removes redundant computations; `licm` hoists loop-invariant code; `loop-unroll` reduces branch overhead; and `dce` cleans newly dead code. This closely matches the most frequent passes in autotuner-discovered optimal lists from [2], validating that the models have internalized effective optimization heuristics through pretraining alone.

G. Failure Mode Classification

Three failure modes are identified:

Type 1 — Reasoning token overflow (reasoning-mode models): The model allocates all 80 output tokens to chain-of-thought content (`<think>...</think>`) and produces no usable pass string. Dominant failure mode for Qwen3.5 reasoning configurations. Mitigated by constrained decoding [9] or non-reasoning mode.

Type 2 — IR hallucination (Nanbeige4.1-3B): The model hallucinates fragments of input LLVM-IR into its output (e.g., `A028484880, align 8`; `@C7 = dso_IR_CONST_...`), producing outputs containing no valid pass names. Dominant failure mode for Nanbeige. Mitigated by catalog-constrained decoding that makes IR-like tokens grammatically invalid.

Type 3 — Structured output collapse (all models, minor): A small fraction of outputs consist of single valid pass names, empty strings, or malformed pipe strings caught by the ≥ 2 pass threshold. Mitigated by more explicit prompt formatting and constrained decoding.

VI. Discussion

A. Reasoning as Liability for Structured Tasks

The result that reasoning mode *harms* pass selection performance is our most counterintuitive finding, and merits careful interpretation. Chain-of-thought reasoning demonstrably improves performance on open-ended mathematical, commonsense, and code-generation tasks [1]. The key distinction is task structure.

Pass selection from LLVM IR is a *constrained pattern retrieval* task: given IR structural features, select the most appropriate pre-specified sequence from a finite catalog. The optimal model behavior is rapid feature-to-pattern association, not extended deliberation. Chain-of-thought introduces overhead that, under a tight token budget, prevents the model from ever reaching the structured answer phase. This insight has a direct engineering implication: for structured compilation tasks, inference should use non-reasoning mode with constrained output decoding [9], reserving reasoning mode for upstream tasks like IR complexity classification or target-constraint interpretation.

B. Instruction Following vs. Parameter Count

The Nanbeige4.1-3B result is striking: the largest model achieves the lowest performance by over two orders of magnitude. This inverts the common assumption that larger models are uniformly better. The Qwen3.5 series was trained with explicit instruction-following objectives (RLHF/DPO alignment), while Nanbeige4.1-3B appears to have weaker structural output compliance—consistently resorting to chain-of-thought even when the prompt explicitly forbids it.

For practitioners integrating LLMs into compiler toolchains, this implies that model *training objective* (instruction compliance) should be a primary selection criterion, not raw parameter count. Benchmarks evaluating structured output compliance (e.g., IFEval [1]) are better predictors of compiler integration performance than general reasoning benchmarks.

C. Quality of Successful Predictions

The 84% compilation success rate for the dominant pipeline (540/642) confirms robustness across the diverse SingleSource file set. The 16% failure rate (102/642) within successful extractions reflects pass-sequencing conflicts specific to certain IR patterns. PassListEval-style pre-validation—verifying each catalog entry against a representative test corpus before deployment—would eliminate these failures.

The near-identical median binary sizes across configurations (16,064–16,112 bytes, <0.3% spread) indicates that within the SingleSource collection, the dominant pipeline achieves size quality comparable to `-O3`. This is a positive result: a training-free model using a seven-pass sequence matches the size output of an aggressively tuned compiler level.

D. Toward a Correctness-Aware Selector Architecture

Our results motivate the following architecture for production deployment:

1) **Catalog construction:** Build a tiered catalog of $\leq$100 pre-validated sequences derived from autotuner output and verified via PassListEval on the target platform. Tier 1 (5–10 entries) covers the large majority of programs.
2) **Constrained generation:** Force model output to a catalog index via grammar-guided decoding [9]. The task becomes multiple-choice classification over the catalog—far more robust than free-form generation.
3) **Backend execution:** Apply the selected catalog entry via the deterministic compiler. On the rare edge-case compilation failure, fall back to `-Oz` with no correctness risk.
4) **Build-system integration:** Cache LLM responses keyed by normalized IR hash. The same function always receives the same pass selection across builds, ensuring reproducibility.

E. Embedded Target Extensions

For ARM Cortex-M4 or RISC-V targets, two modifications are needed. First, the catalog must exclude size-increasing transformations (`loop-unroll` with high factors, vectorization) and include size-reducing passes (`iroutliner`, `mergefunc`). Second, target constraints must be communicated via the prompt: "optimize for code size on a Cortex-M4 with 256 KB Flash." LLM inference occurs at compile time on the host, not on the target—1.71–2.58 s per file is compatible with standard cross-compilation workflows and CI/CD pipelines.

The multi-dimensional embedded optimization objective:

$$\text{Score}(p) = \alpha\frac{\text{CodeSize}(p)}{\text{Flash}} + \beta\frac{\text{Runtime}(p)}{\text{Deadline}} + \gamma\frac{\text{Energy}(p)}{\text{Budget}} \quad (2)$$

requires execution time and energy measurement infrastructure not present in the current framework—both are priorities for future work.

VII. Limitations and Future Work

A. Evaluation Limitations

Single-trial evaluation: All measurements are single-trial. A rigorous study should report mean, standard deviation, best, and worst over at least 5 trials per file per configuration to characterize LLM output stochasticity.

Missing per-file regression: The per-file LLM-vs-`-O3` scatter is unrealizable in the current implementation (Figure **??**). Correcting this through file-keyed CSV joins is the single most important engineering improvement needed.

No execution time or energy: Binary size is the sole quality proxy. Integrating QEMU-based execution simulation and architecture-specific energy estimation (Eq. 2) is required for embedded system claims.

GPU constraint: The 8 GB VRAM budget limits models to $\leq$3B parameters and IR context to 2,000 characters. Higher-VRAM or cloud configurations would unlock 7B–13B models and 16,384-token context windows, likely improving success rates substantially.

B. Planned Improvements

Constrained decoding: Implementing Outlines [9] or similar grammar-guided decoding would enforce structured output at the token level, eliminating Type 1 and Type 3 failures. Expected improvement: success rate for non-reasoning Qwen3.5 from 57% to >90%; reasoning configurations may reach 50%+.

Catalog expansion: Expanding the 17-pass whitelist to include module-level passes (`globalopt`, `iroutliner`, `deadargelim`), CGSCC passes (`inline`, `argpromotion`), and additional loop passes (`loop-rotate`, `indvars`, `loop-deletion`) would enable richer suggested pipelines.

IR normalization: Implementing Cummins et al. [3] normalization—replacing register names with canonical placeholders, stripping debug metadata—would improve model generalization for the same character budget.

Embedded target evaluation: Cross-compilation to `arm-none-eabi` (Cortex-M4) and RISC-V with target-specific prompt engineering is the primary experimental extension.

Larger models via API: Cloud-hosted 70B+ models (Llama 3.3, GPT-4o, Claude Sonnet) with full IR context windows would test whether scale overcomes the structured-output gap identified here.

VIII. Conclusion

We have presented the first systematic empirical evaluation of small-scale ($\leq$3B parameter) instruction-tuned LLMs as training-free LLVM optimization pass selectors on 972 files from the LLVM test suite. Our key conclusions are:

1) **Non-reasoning mode substantially outperforms reasoning mode** for structured pass selection. Chain-of-thought verbosity consumes output token budgets before the structured answer is produced, dropping success rates from 55–57% to 8.7–13.6%.
2) **Instruction-following fidelity dominates parameter count** at sub-3B scale. Nanbeige4.1-3B (largest tested) achieves 0.4% success vs. 57% for Qwen3.5-0.8B. Architecture and RLHF alignment matter more than raw scale.
3) **Successful predictions converge on high-quality, consistent pipelines.** Both Qwen3.5 non-reasoning variants independently select the same seven-pass sequence (`mem2reg`, `instcombine`, `simplifycfg`, `gvn`, `licm`, `loop-unroll`, `dce`), matching autotuner-discovered optimal patterns from systems trained on 22 billion compilations.
4) **Binary size quality is comparable to `-O3`** for successful configurations, with median sizes of $\approx$16 KB and compile latencies below 0.10 s.
5) **Three addressable failure modes** account for the 43% failure gap: reasoning token overflow (Type 1), IR hallucination (Type 2), and structured-output collapse (Type 3). Constrained decoding resolves Types 1 and 3; longer IR context and improved normalization address Type 2.
6) **The training-free LLM-as-selector paradigm is viable at sub-3B scale.** Zero-shot instruction following on a consumer GPU can identify effective optimization sequences from raw LLVM-IR without any compiler-specific training, establishing a foundation for correctness-aware, catalog-based compiler optimization frameworks deployable within standard cross-compilation workflows.

Acknowledgment

The authors thank the LLVM community for open-source compiler infrastructure and the Hugging Face community for open model hosting. Experiments were conducted on an NVIDIA RTX 4060 under the VIT Compiler Design course infrastructure.

References

[1] Z. Zhang et al., "The new compiler stack: A survey on the synergy of LLMs and compilers," *arXiv:2601.02045*, 2026.

[2] C. Cummins, V. Seeker, D. Grubisic, B. Rozière, J. Gehring, G. Synnaeve, and H. Leather, "Meta large language model compiler: Foundation models of compiler optimization," *arXiv:2407.02524*, 2024.

[3] C. Cummins, V. Seeker, D. Grubisic, M. Elhoushi, B. Rozière, J. Gehring, F. Gloeckle, K. Hazelwood, G. Synnaeve, H. Leather, and Y. Liang, "Large language models for compiler optimization," *arXiv:2309.07062*, 2023.

[4] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proc. CGO*, pp. 75–86, 2004.

[5] A. Haj-Ali, Q. Huang, W. Moses, J. Xiang, J. Wawrzynek, K. Asanovic, and I. Stoica, "AutoPhase: Juggling HLS phase orderings in random forests with deep reinforcement learning," in *Proc. MLSys*, 2020.

[6] Y. Liang et al., "Coreset-NVP: Neural value prediction for compiler pass ordering," *arXiv*, 2023.

[7] A. Author et al., "LEGO-Compiler: Enhancing neural compilation through translation composability," *arXiv:2505.20356*, 2025.

[8] LLVM Project, "LLVM test suite," https://github.com/llvm/llvm-test-suite, 2024.

[9] R. Willard et al., "Outlines: Structured text generation," https://github.com/outlines-dev/outlines, 2023.

[10] C. Chen et al., "Static neural compiler optimization via deep reinforcement learning," *arXiv:2008.08951*, 2020.

[11] Y. Li, "Optimizing LLVM pass list using reinforcement learning," MPhil Thesis, University of Cambridge, 2022.